NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 6

TOPIC : FINDING ROOTS USING THE ITERATION METHOD
COURSE CODE : MATH 0541-2212
COURSE TITLE : NUMERICAL ANALYSIS SESSIONAL

Submitted By:
MD. TANVIR RAHMAN PRANTO
0712340012101005
CSE -26

Submitted To:
SAIFUR RAHMAN
LECTURER, CSE

Finding Roots Using the Iteration Method:

Theory:

The Iteration Method, also known as Fixed Point Iteration, is a numerical technique used to find the roots of an equation. The basic idea is to transform the original equation into a form where the root can be approximated through successive iterations.

Concept:

- **Original Equation**: Start with an equation of the form $x^3 = 1 - x^2$.
- **Rearranged Form**: Rearrange the equation into $x = g(x)$, such as $g(x) = \frac{1}{\sqrt{x+1}}$. This form allows us to iteratively update the value of x.

Steps:

1. **Initial Guess**: Start with an initial guess x_0.
2. **Iteration**: Compute the next approximation using $x_{n+1} = g(x_n)$.
3. **Convergence**: Repeat the process until the difference between successive approximations is less than a specified tolerance.

Python Code: The provided Python code implements this method. It defines the function $g(x)$, takes an initial guess and a tolerance as inputs, and iteratively updates the value of x until the desired accuracy is achieved. The code prints each step, showing the progress towards the root.

Algorithm:

Algorithm for Finding Roots Using the Iteration Method

1. Start

2. Import the math module
- Import the math module to use the square root function.

3. Define the function for iteration
- Define the function `f(x)` which is the rearranged form of the original equation for iteration:

f(x) = 1 / sqrt(x + 1)

4. Initialize variables
- Set the initial guess `x0` to 0.
- Set the tolerance `tolerance` to 10^-4.
- Initialize the iteration counter `i` to 1.

5. Compute the first iteration

- Calculate `x = f(x0)`.
- Print the current step and value of `x`.

6. Iterative process

- While the absolute difference between the new and previous values (`|x - x0|`) is greater than the tolerance:
  - Increment the iteration counter `i`.
  - Update `x0` to the current value of `x`.
  - Compute the new value of `x` as `x = f(x0)`.
  - Print the current step and value of `x`.

7. Output the final result

- Print the final value of `x` when the tolerance condition is met.

8. End

Code:

```python
import math

# Given function x^3 = 1 - x^2 rearranged as
# f(x) = 1 / sqrt(x + 1) for iteration
def f(x):
    return 1 / math.sqrt(x + 1)

# First approximation
x0 = 0

# Tolerance
tolerance = 10 ** -4

# Iteration count
i = 1

# Initial iteration
x = f(x0)
print(f"x{i} = {x:.6f}")

# Iterative process
while abs(x - x0) >= tolerance:
    i += 1
    x0 = x
    x = f(x0)
    print(f"x{i} = {x:.6f}")

print(f"Final x{i} = {x:.6f}")
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/
x1 = 1.000000
x2 = 0.707107
x3 = 0.765367
x4 = 0.752632
x5 = 0.755361
x6 = 0.754774
x7 = 0.754900
x8 = 0.754873
Final x8 = 0.754873
PS C:\Users\Lenovo>
```

Conclusion: This Python code uses the Iteration Method to find the root of an equation. It iteratively updates the value of (x) using a transformed function until the difference between successive approximations is within a specified tolerance. The code's simplicity makes it an excellent tool for understanding numerical root-finding techniques.